Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 8 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

1. Week 8 Overview

Week 8 focused on **Pipeline as Code and Server Deployment** for the Automated Grocery Inventory Dashboard.

The existing Jenkins Continuous Integration setup from Week 7 was extended into a Jenkins Pipeline using a **Jenkinsfile**. The pipeline performs source checkout, application testing, WAR packaging, and deployment to an external Apache Tomcat server.

The application was converted from a Spring Boot executable JAR to a WAR deployment so that it could run on Tomcat.

The major objectives completed during this week were:

- Create a Jenkinsfile for Pipeline as Code.
- Implement Checkout, Build, Package and Deploy stages.
- Convert the Spring Boot application to WAR packaging.
- Configure Apache Tomcat 10.1 for deployment.
- Deploy the application automatically through Jenkins.
- Parameterize the deployment using the `APP_ENV` parameter.
- Verify the deployed application through its Tomcat URL.

2. Technology Stack

- **Java:** 21
- **Spring Boot:** 4.1.0
- **Maven:** Maven Wrapper
- **MySQL:** Grocery Inventory database
- **Jenkins:** Pipeline as Code
- **Apache Tomcat:** 10.1.59
- **Git/GitHub:** Source code management

Tomcat was configured to run on port **8082**, while Jenkins continued to use port **8080**.

3. WAR Deployment Configuration

The Spring Boot application was originally configured as an executable JAR. For deployment to an external Tomcat server, the Maven packaging was changed to WAR.

The following packaging was added to `pom.xml`:

```
<packaging>war</packaging>
```

Tomcat was added as a provided dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
```

```
    <scope>provided</scope>
</dependency>
```

The main Spring Boot application class was extended using `SpringBootServletInitializer` so that the application could run inside an external servlet container.

The WAR was successfully generated using:

```
./mvnw clean package
```

The generated artifact was:

```
automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.war
```

The main Spring Boot application class was extended using `SpringBootServletInitializer` so that the application could run inside an external servlet container.

The WAR was successfully generated using:

```
./mvnw clean package
```

The generated artifact was:

```
automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.war
```

The WAR size was approximately 51 MB.

4. Tomcat Server Deployment

Apache Tomcat 10.1.59 was installed using Homebrew and configured to run on port 8082 to avoid conflict with Jenkins.

The Tomcat deployment directory used was:

```
/opt/homebrew/opt/tomcat@10/libexec/webapps
```

The WAR was deployed using the application context name:

```
automated-grocery-inventory-dashboard.war
```

Tomcat successfully detected and unpacked the WAR.

The deployed application was accessed using:

```
http://localhost:8082/automated-grocery-inventory-dashboard/items
```

The application returned HTTP status **200**, confirming successful deployment.

During deployment, application links were also updated to use Thymeleaf context-aware URLs so that pages such as `/items`, `/transactions`, and `/alerts` work correctly under the Tomcat application context.

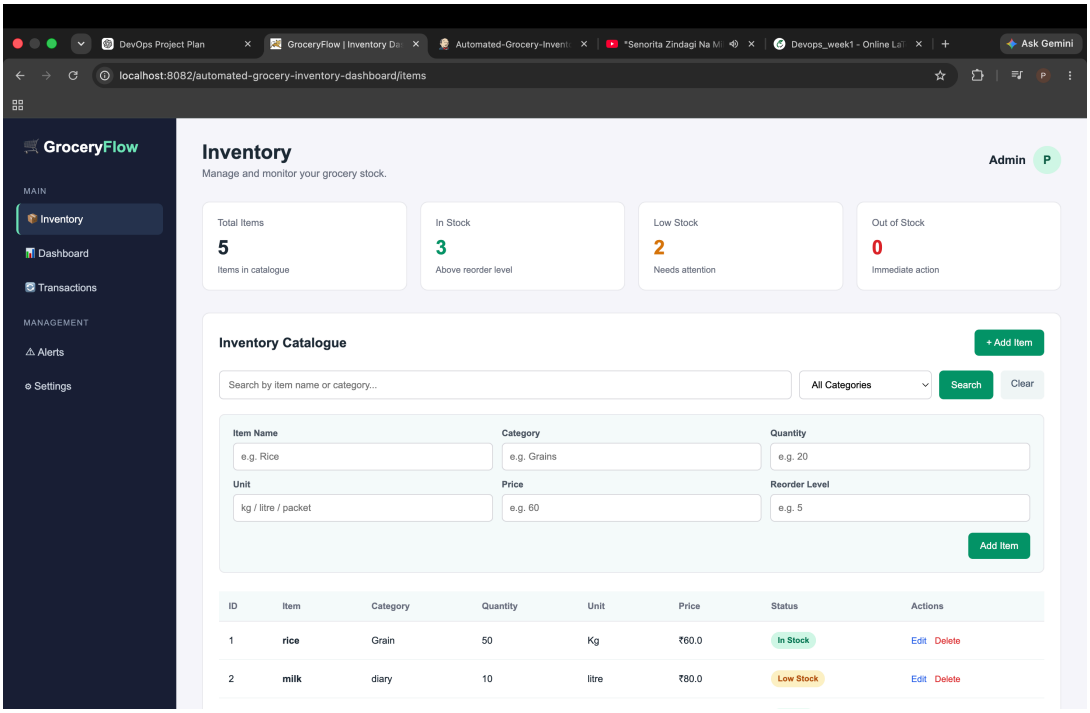


Figure 1: Automated Grocery Inventory Dashboard deployed on Tomcat

5. Jenkins Pipeline as Code

A `Jenkinsfile` was created in the project repository. Jenkins reads this file directly from the GitHub feature branch.

The pipeline contains four main stages:

1. **Checkout** – retrieves the source code from GitHub.
2. **Build** – configures Java 21 and runs Maven tests.
3. **Package** – creates the deployable WAR artifact.
4. **Deploy** – copies the WAR into the Tomcat webapps directory.

The pipeline also contains the parameter:

$$\text{APP_ENV} = \text{development} \;/\; \text{production}$$

For the successful deployment run, the selected environment was:

$$\text{APP_ENV} = \text{development}$$

The deployment stage displayed the selected environment in the Jenkins console, providing evidence that the pipeline parameter was being used.

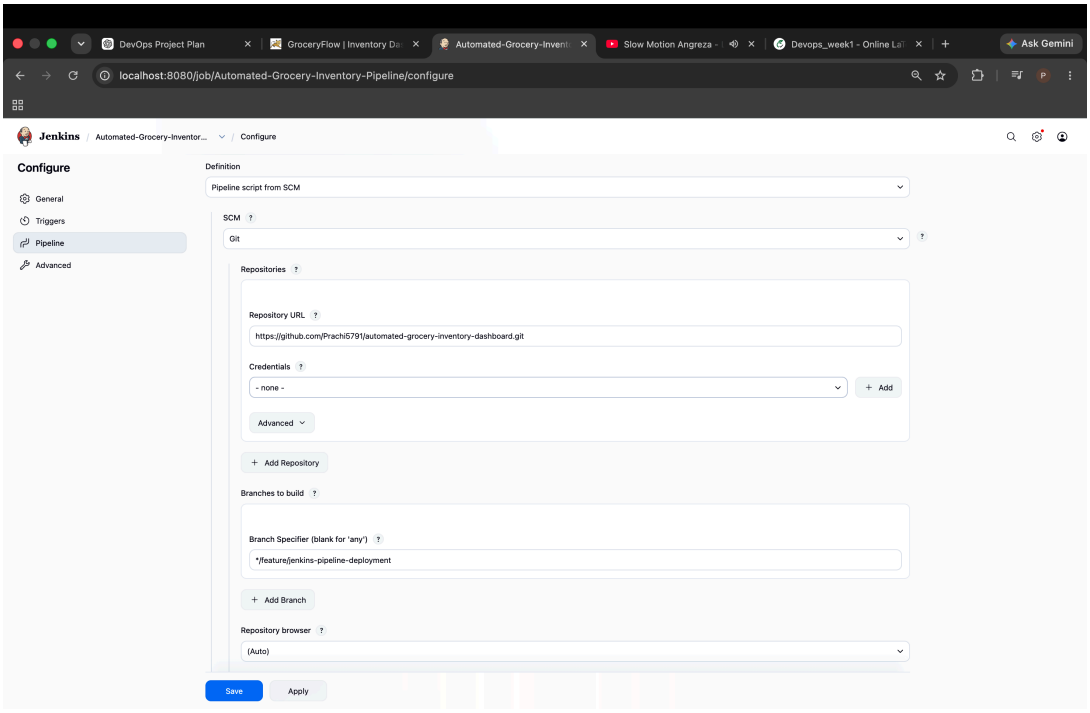


Figure 2: Jenkins Pipeline configured to use Jenkinsfile from GitHub

6. Pipeline Execution and Results

The Jenkins Pipeline was executed using the `feature/jenkins-pipeline-deployment` branch.

The Build stage successfully executed Java 21 and Maven tests.

The test result was:

```
Tests run: 1
Failures: 0
Errors: 0
Skipped: 0
BUILD SUCCESS
```

The Package stage successfully generated the WAR:

```
automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.war
```

The Deploy stage copied the WAR into the Tomcat deployment directory.

The final pipeline result was:

```
Pipeline completed successfully for environment: development
Finished: SUCCESS
```



Figure 3: Successful Jenkins Pipeline execution

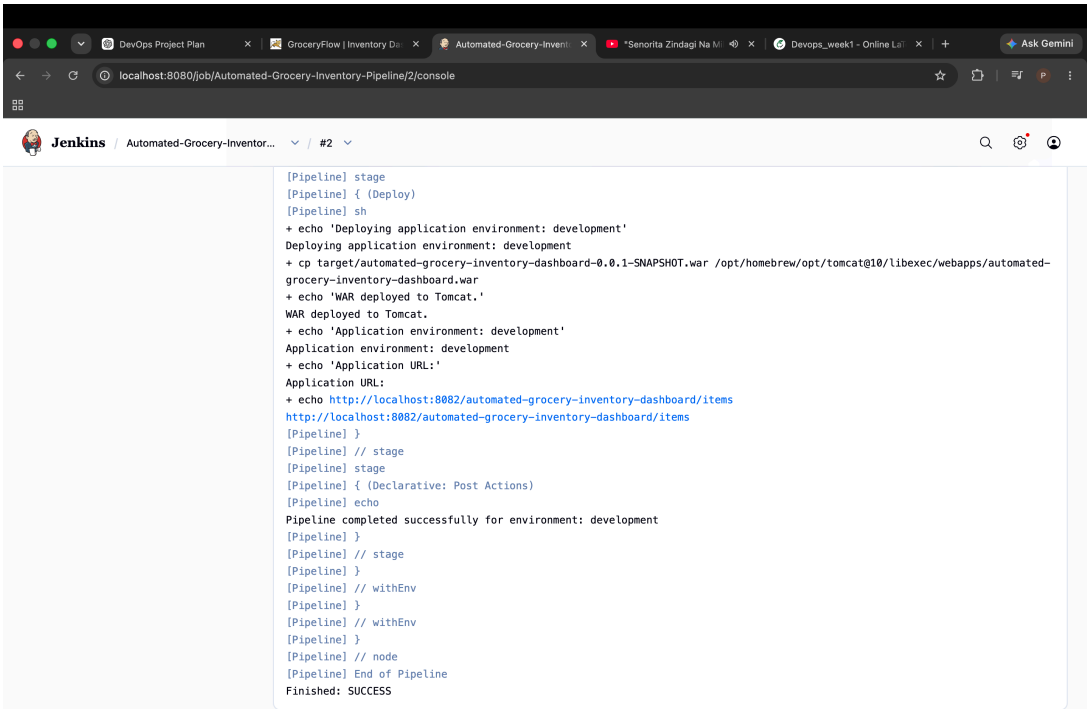


Figure 4: Jenkins deployment stage and environment parameter

7. Final Pipeline Flow

The completed Week 8 deployment workflow is:

GitHub → Jenkinsfile → Checkout → Build & Test → Package WAR → Deploy to Tomcat → Running Application

The final deployed URL is:

http://localhost:8082/automated-grocery-inventory-dashboard/items

This provides an automated path from source code to a deployed application.

8. Issues and Resolutions

During deployment, the application initially returned HTTP 404 for URLs such as `/alerts`. The issue occurred because the application was running under the Tomcat context path `/automated-grocery-inventory-dashboard`.

The template links were changed from absolute paths such as:

href="/alerts"

to Thymeleaf context-aware paths:

th:href="@{/alerts}"

The same correction was applied to the relevant inventory and transaction links. After redeployment, the application pages were accessible through the Tomcat context path.

9. Week 8 Deliverable Status

Deliverable	**Status**
Jenkinsfile	Completed
Checkout stage	Completed
Build stage	Completed
Package WAR stage	Completed
Tomcat deployment stage	Completed
Parameterized environment	Completed
Successful pipeline run	Completed
Deployed application URL	Verified
Configuration evidence	Captured

10. Conclusion

Week 8 successfully extended the Grocery Inventory Dashboard from a Continuous Integration setup into a deployment pipeline using Jenkins Pipeline as Code.

The application was converted to WAR packaging, deployed on Apache Tomcat, and integrated with a Jenkinsfile containing Checkout, Build, Package and Deploy stages. The pipeline successfully executed Maven tests, generated the WAR artifact, deployed it to Tomcat, and reported a successful build.

The `APP_ENV` parameter provided environment-specific deployment selection, while the deployed application was verified through the Tomcat URL.

Therefore, the project now demonstrates an automated CI/CD workflow from GitHub source code to a running server application.